

Einführung in die Programmierung (WIN+BIT)

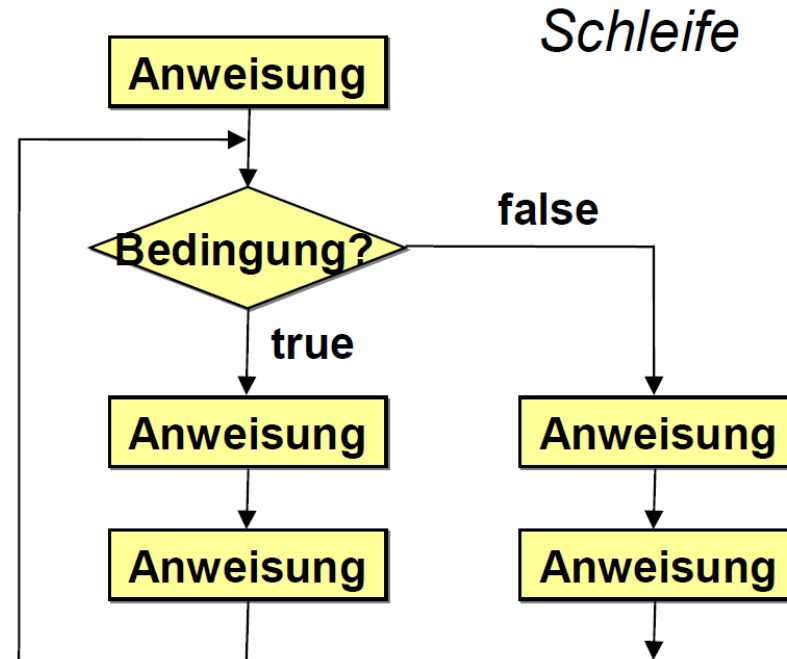
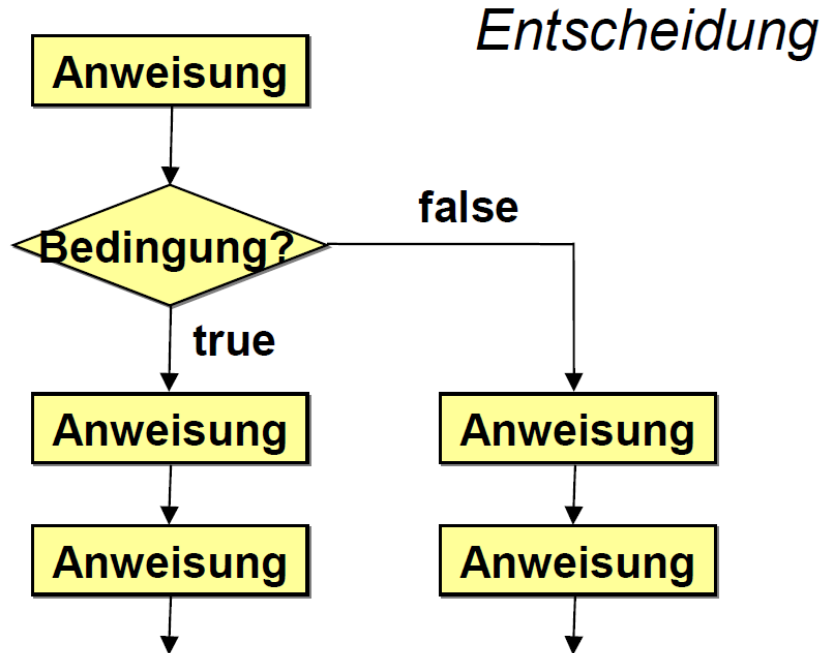
Kontrollstrukturen: Schleifen

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



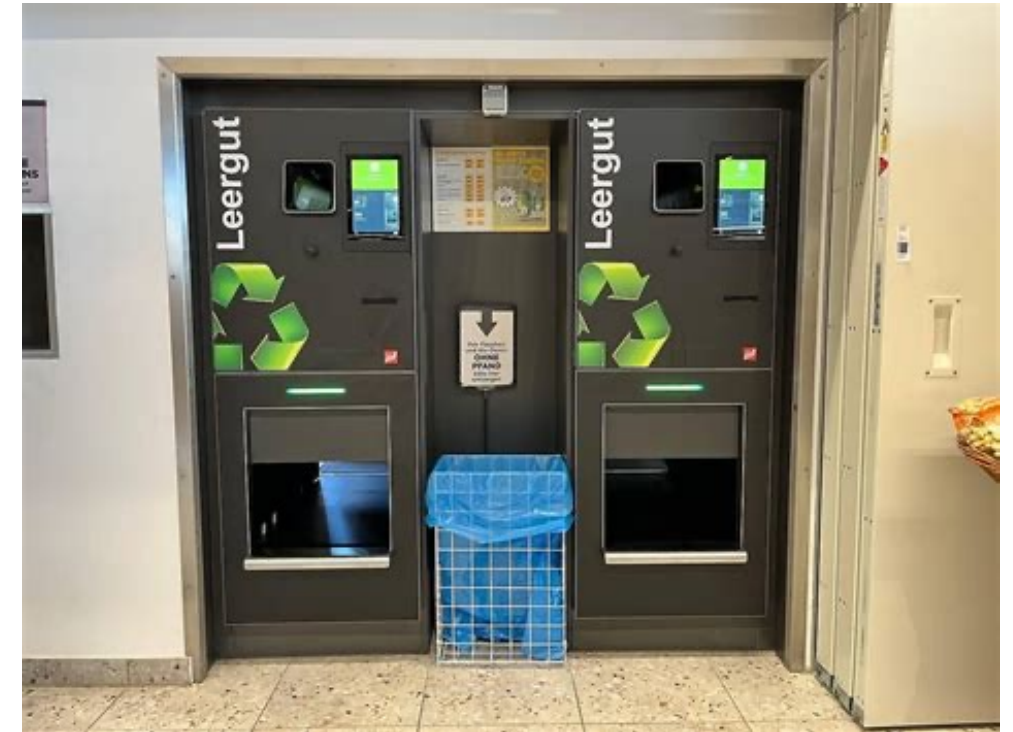
Was sind Kontrollstrukturen?

- Kontrollstrukturen erlauben es, den Ablauf eines Programms zu verändern (statt Zeile für Zeile).
- In Abhängigkeit von Bedingungen können bestimmte Programmteile ausgeführt werden.



Motivation

- Ein Pfandautomat nimmt folgende Gegenstände entgegen und druckt Pfandbons:
 - Mehrweg-Glasflaschen: 8 cent pro Stück
 - Einwegflaschen (Glas und Plastik): 25 cent pro Stück
 - Dosen: 25 cent pro Stück
 - Getränkekästen (leer): 3 Euro pro Stück
- Fragestellung:
 - Wie kann ich den Betrag für den Pfandbon berechnen?
 - Wie kann ich eine Warnung ausgeben, wenn der Lagerbehälter für Dosen seine Füllgrenze von 1000 Dosen überschritten hat?



Modellierung

- Wenn man die Bon-Taste drückt, liefert der Pfandautomat eine Zeichenkette, darin steht:
 - ‘M’ für eine Mehrweg-Glasflasche
 - ‘E’ für eine Einwegflasche (Glas und Plastik)
 - ‘D` für eine Dose
 - ‘K’ für einen Getränkekästen (leer)
- Beispiele:
 - “MMEDK“
 - “DDDDDDDE“,
 - “MMMMMMMMMMMMMKD“

Wdh: If



code05.
PfandLoop

- Mit der Methode getPfand() kann das Pfand für einen Gegenstand berechnet werden.
- Wie kann man nun das Pfand für mehrere Gegenstände, z.B. “KFFFPDDDP” berechnen?

```
double result = 0.0;
if (c == 'M') {
    result = 0.08;
}
if (c == 'E') {
    result = 0.25;
}
if (c == 'D') {
    result = 0.25;
}
if (c == 'K') {
    result = 3;
}
betrag += result;
```

Schleifen

- Schleifen erlauben es, einen Block (Folge von Anweisungen) **mehrfach** zu durchlaufen
- Java kennt 4 Arten von Schleifen:
 - for-Schleifen
 - while-Schleifen
 - do-while-Schleifen
 - for-each-Schleifen (machen wir später)
- Alle Schleifen haben eine Abbruchbedingung, d.h. sind darauf angelegt, zu **terminieren**

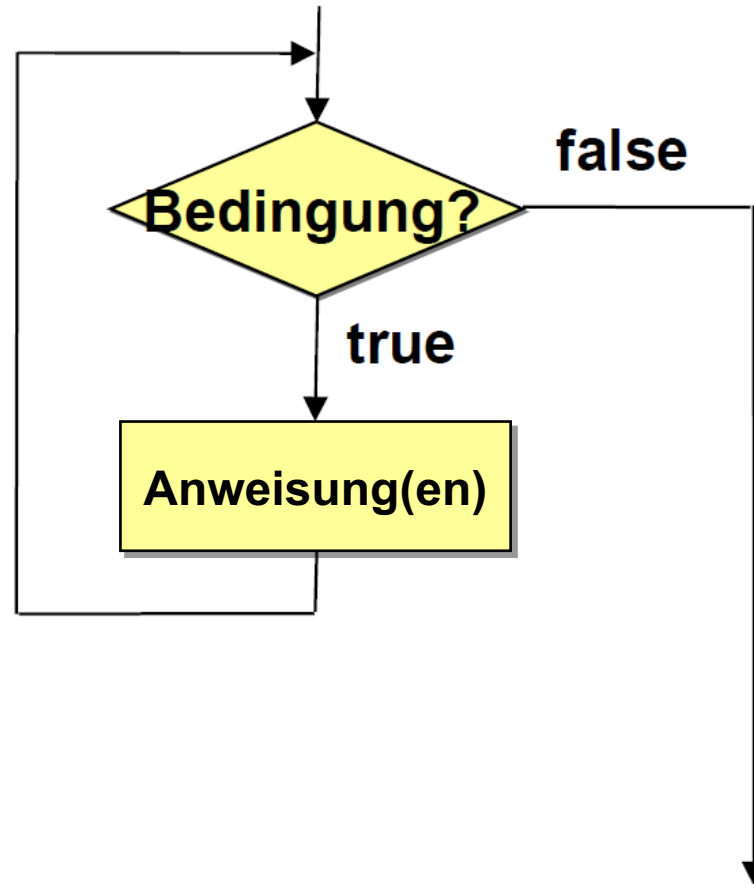
while-Schleife

- Allgemeine Schleifenform
- Bedingung wird **vor** erster Ausführung des Anweisungsblocks geprüft

```
while (<Bedingung>) {  
    <Anweisung(en)>  
}
```

1. <Bedingung> auswerten
 - A. Bedingung false → Schleife abbrechen
 - B. Bedingung true → <Anweisung(en)> ausführen, dann weiter mit 1.

while-Schleife



while-Schleife

```
while (<Bedingung>) {  
    <Anweisung(en)>  
}
```

- Schleifenvariable muss außerhalb der Schleife deklariert und im **Schleifenblock** aktualisiert werden

Anwendung von Schleifen zur Pfandberechnung



code05.
PfandLoop

Return Type	Method	Description
int	length()	Returns the length of this string.
char	charAt(int index)	Returns the char value at the specified index.

```
String eingabe = ...  
double betrag = 0.0;  
int index = 0;
```

Initialisierung einer Variable
Schleifenbedingung

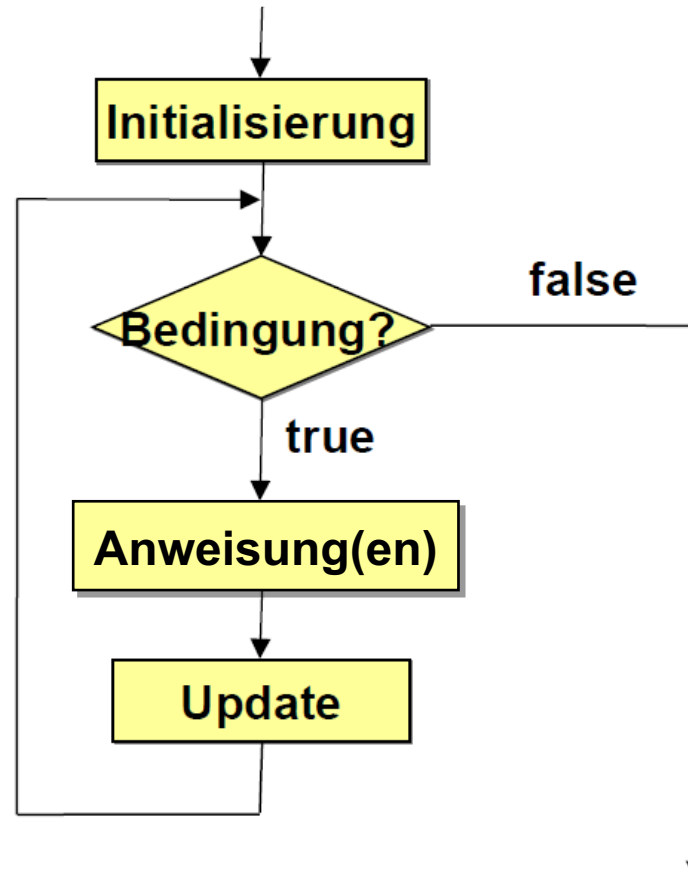
```
while (index < eingabe.length()) {  
    char c = eingabe.charAt(index);  
    ...  
    index++;  
}
```

Anweisungen

```
System.out.println(betrag);  
}
```

Variablen-Aktualisierung
(Update)

Typische Anwendung (allgemein)



for-Schleife

```
for (<Initialisierung>; <Bedingung>; <Update>) {  
    <Anweisung(en)>  
}
```

1. <Initialisierung>, Deklaration einer Schleifenvariablen, z.B. `int x = 1`
2. <Bedingung> auswerten, z.B. `x <= 20`
 - A. Bedingung false → Schleife abbrechen
 - B. Bedingung true → <Anweisung(en)> ausführen, dann <Update> ausführen (z.B. `x++`), dann weiter mit 2.

Anwendung: Quadratzahlen ausgeben

- Wollen die Quadratzahlen von 0^2 bis n^2 ausgeben

```
int i = 0;
System.out.println(i + "² = " + (i * i));
i++;
System.out.println(i + "² = " + (i * i));
i++;
System.out.println(i + "² = " + (i * i));
i++;
// ... (insgesamt n-mal)
```

- umständlich, vor allem für große Tabellen (große Wert für n)



code05.
Quadratic
Numbers

Anwendung: Quadratzahlen ausgeben

– mit while-Schleife:

```
int n = 10;
int i = 0;
while (i <= n) {
    System.out.println(i + "² = " + (i * i));
    i++;
}
```

– mit for-Schleife:

```
for (int i = 0; i <= n; i++) {
    System.out.println(i + "² = " + (i * i));
}
```



code05.
Quadratic
Numbers

Einfache Inkrement-for-Schleife

```
for (int i = 0; i < 10; i++) {  
    // Anweisung(en)  
}
```

- Häufigste Anwendung einer for-Schleife
- Lies *"Führe Anweisungen für alle Werte i von 0 (inklusive) bis 10 (exklusive) aus"*

```
for (int i = 0; i <= 10; i++) {  
    // Anweisung(en)  
}
```

- Lies *"Führe Anweisungen für alle Werte i von 0 (inklusive) bis 10 (inklusive) aus"*



code05.
ForLoops

Weitere for-Loops

- Weitere Beispiele für Schleifen mit anderen Inkrement-Anweisungen:

```
for (double d = -1; d <= 1; d = d + 0.1) {  
    System.out.printf("%5.1f%n", d);  
}  
  
for (int i = 1; i <= 1024; i = 2 * i) {  
    System.out.println(i);  
}  
  
for (String s = ""; s.length() < 10; s = s + "a") {  
    System.out.println(s);  
}
```

- Vorher überlegen: Was passiert hier?



code05.
ForLoops2

for-Schleife in einer for-Schleife

```
for (<Initialisierung>; <Bedingung>; <Update>) {  
    for (...) {  
        <Anweisung(en)>  
    }  
}
```

- Schleifen kann man auch schachteln
- Geht auch
 - mit while-Schleifen
 - mit Mischformen (for-Schleife innerhalb while-Schleife, ...)
 - mit mehr als zwei Ebenen (wird aber schnell unübersichtlich!)



code05.
ForLoops
Nested

for-Schleife zu while-Schleife

```
for (<Initialisierung>; <Bedingung>; <Update>) {  
    <Anweisung(en)>  
}
```



```
<Initialisierung>;  
while (<Bedingung>) {  
    <Anweisung(en)>;  
    <Update>;  
}
```

- Jede for-Schleife kann durch eine while-Schleife ersetzt werden



code05.
WhileLoops

while-Schleife zu for-Schleife

- while-Schleifen dieser Form sollte man durch for-Schleifen ersetzen, da der Code dadurch übersichtlicher wird

```
<Initialisierung>;  
while (<Bedingung>) {  
    <Anweisung(en)>;  
    <Update>;  
}
```



```
for (<Initialisierung>; <Bedingung>; <Update>) {  
    <Anweisung(en)>  
}
```



code05.
WhileLoops

Randbemerkung

Jede while-Schleife lässt sich als for-Schleife schreiben, indem man einfach

```
while (<Bedingung>) {  
    <Anweisung(en)>;  
}
```

durch Initialisierung und Update sind leer gelassen

```
for (; <Bedingung>;) {  
    <Anweisung(en)>  
}
```

ersetzt. Dies ist aber eher eine theoretische Betrachtung: Programmiersprachen mit while ohne for sind gleich-"mächtig" wie Sprachen ohne while mit for. In der Praxis verwirrt die untere Schreibweise eher.

do-while-Schleife

- Allgemeine Schleifenform
- Bedingung wird **nach** erster Ausführung des Anweisungsblocks geprüft (anders als bei while!)

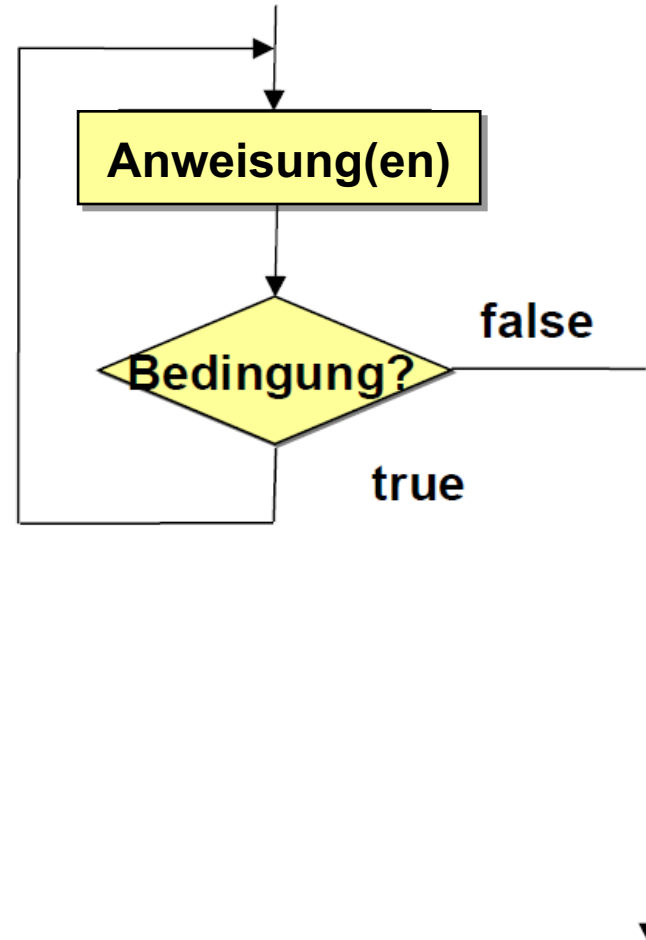
```
do {  
    <Anweisung(en)>  
} while (<Bedingung>);
```

1. <Anweisung> ausführen
2. <Bedingung> auswerten
 - A. Bedingung false → Schleife abbrechen
 - B. Bedingung true → weiter mit 1.



code05.
DoWhile

do-while-Schleife



Endlos-Schleife

- Falls Schleifenbedingung immer true ist, **terminiert** die Schleife **nicht**
- Absichtlich
 - Warten auf **Ereigniseintritt**, Abbruch mit **break**
(eleganter als while bzw. do-while)
- Unabsichtlich
 - **Programmierfehler**
 - Änderung der Schleifenvariable im Schleifenrumpf vergessen

```
int id = 0;
while (id < 10) {
    System.out.println("id: " + id);
}
```



code05.
EndlessLoop1

Endlos-Schleife?!?

```
// Summiere die Zahlen 0, 0.1, 0.2, ..., 0.9 auf  
double sum = 0.0;  
for (double d = 0.0; d != 1.0; d = d + 0.1) {  
    sum = sum + d;  
}  
System.out.println("Summe: " + sum);
```

- Was passiert hier?
- Programm hat einen (unbeabsichtigten) Fehler
 - "Bug"
- Wie findet man den?



code05.
EndlessLoop2

Fehlersuche...

```
// Summiere die Zahlen 0, 0.1, 0.2, ..., 0.9 auf  
double sum = 0.0;  
for (double d = 0; d != 1; d = d + 0.1) {  
    System.out.println("d = " + d);  
    sum = sum + d;  
    System.out.printf("sum = %d%n", sum);  
}  
System.out.println("Summe: " + sum);
```

- Naheliegender Ansatz: Print-Statements einbauen ("printf-Debugging")
 - Nicht sehr elegant
 - führt zu viel Ausgabe
 - Programm läuft immer weiter
 - man muss dran denken, die Ausgaben wieder zu entfernen
- **Besser: Debugger benutzen!**

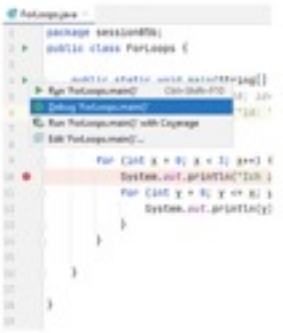


code05.
EndlessLoop3

Erinnerung: Debugger

Debugger und Breakpoints

- Debugging: Finden und Entfernen von "Bugs" (Fehlern) im Programm
- Nachverfolgen des Kontrollflusses
- Breakpoint weist die Laufzeitumgebung an, die Ausführung an dieser Stelle anzuhalten, die Ausführung wird "unterbrochen"
- IntelliJ:
 - Strg + F8 zum Breakpoint Setzen/Entfernen
 - Klick neben main-Methode → Debug oder
 - Debug-Tool-Window
 - F8 Einzelschrittausführung nach Halt
 - F9 normal weiter bis zum nächsten Breakpoint

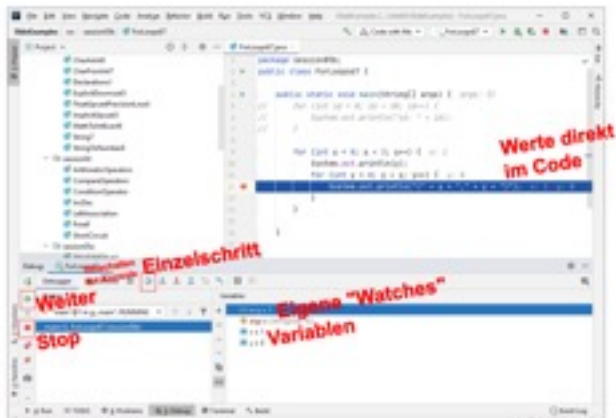


IntelliJ IDEA interface showing a Java file with a breakpoint set on a line of code.

Einführung in die Programmierung (WIN+BIT), Prof. Dr. Johannes C. Schneider

Hausaufgabe für Sie:

- Öffnen Sie das material-Projekt und debuggen Sie in die Code-Beispiele dieser und der letzten Vorlesungen
- Vollziehen Sie nach, wie Variablen und die Schleifenbedingungen sich verändern



IntelliJ IDEA interface showing the Debug console and Watches window. Red handwritten text annotations are present: "Werte direkt im Code" (Values directly in the code) pointing to the code editor, "Einzelschritt" (Step) pointing to the Debug console, "Weiter" (Next) pointing to the Debug console, "Stop" pointing to the Debug console, and "Eigene 'Watches' Variablen" (Own 'Watches' variables) pointing to the Watches window.

- code07
 - BreakContinue1
 - BreakContinue2
 - BreakContinue3
 - DoWhile
 - DrawCoordinatesFor
 - DrawCoordinatesWhile
 - DrawFun
 - DrawLecture
 - EndlessLoop
 - EndlessLoop2
 - ForLoops
 - ForLoops2
 - Kickers1
 - Kickers1For
 - Kickers2a
 - Kickers2b
 - WhileLoops

SlideExamples [...\IntelliJ\SlideExamples] - ForLoops67.java

SlideExamples > src > session05b > ForLoops67

Project

- CharAsInt0
- CharFromInt7
- Declarations1
- ExplicitDowncast5
- FloatUpcastPrecisionLoss4
- ImplicitUpcast3
- MathToIntExact6
- String7
- StringToNumber8
- session04
 - ArithmeticOperators
 - CompareOperators
 - ConditionOperator
 - IncDec
 - LeftAssociation
 - PrintF
 - ShortCircuit
- session05a
 - AbsoluteValue2

ForLoops67.java

```
1 package session05b;
2 public class ForLoops67 {
3
4     public static void main(String[] args) { args: {}
5         // for (int id = 0; id < 10; id++) {
6         //     System.out.println("id: " + id);
7         // }
8
9         for (int x = 0; x < 3; x++) { x: 1
10             System.out.println(x);
11             for (int y = 0; y < x; y++) { y: 0
12                 System.out.println("(" + x + "," + y + ")"); x: 1 y: 0
13             }
14         }
15     }
16 }
17
```

Werte direkt im Code

Umschalten zur Konsole

Debugger

Frames

- "main"@1 in g...main: RUNNING
- main:12, ForLoops67 (session05b)

Variables

- x+y = 1
- args = {String[0]@792}
- x = 1
- y = 0

Weiter

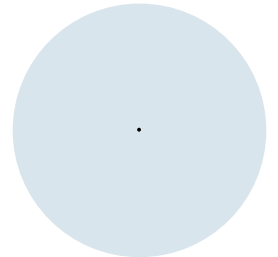
Stop

Eigene "Watches" Variablen

4: Run 6: Problems 5: Debug Terminal Build Event Log

All files are up-to-date (a minute ago) 12:1 CRLF UTF-8 Tab*

Schleifen-Kontrollanweisungen



Sprünge

- **break**
 - Verlässt eine Schleife (vorzeitiger Abbruch oder bei absichtlicher Endlosschleife)
- **continue**
 - Bricht die Ausführung des Schleifenrumpfes ab und springt zum Schleifenkopf

break

```
int i = 0;
while (true) {
    if (i == 5) {
        break;
    }
    System.out.println(i);
    i++;
}
```

- Was passiert?
 - Zahlen 0 bis 4 werden ausgegeben!



continue

```
for (int i = -5; i <= 5; i++) {  
    if (i == 0) {  
        continue;  
    }  
    System.out.println(1.0 / i);  
}
```

- Was passiert?
 - Zahlen 1 / -5 bis 1 / 5 werden ausgegeben, ohne 1 / 0.



code05.
Continue

break bei verschachtelter Schleife

```
int y;  
for (int x = 0; x < 4; x++) {  
    y = 0;  
    while (true) {  
        System.out.println("(" + x + "," + y + ")");  
        y++;  
        if (y >= 3) {  
            break;  
        }  
    }  
}
```

- break springt aus **innerster** Schleife (while)
- **äußere** Schleife wird fortgesetzt



Beispiel mit break und continue

- Typischer Anwendungsfall:
 - Eine Endlosschleife `while (true) { ... }` wird bei bestimmten Benutzereingaben über `break` verlassen.

```
while (true) {  
    System.out.println("Durch welche Zahl soll 100 geteilt "  
                        + "werden? ('e' zum Beenden)");  
    String input = scanner.next();  
    if (input.equals("e")) {  
        break;  
    }  
    // weiterer Code  
}
```



break und continue sind oft nicht nötig

- In vielen Fällen lässt sich der Kontrollfluss eleganter steuern (z.B. durch erweiterte Schleifenbedingung oder andere Formulierung der Schleife)

```
int i = 0;
while (true) {
    if (i == 5) {
        break;
    }
    System.out.println(i);
    i++;
}
```



```
i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```



```
for (i = 0; i < 5; i++) {
    System.out.println(i);
}
```

```
for (int i = -5; i <= 5; i++) {
    if (i == 0) {
        continue;
    }
    System.out.println(1.0 / i);
}
```



```
for (int i = -5; i <= 5; i++) {
    if (i != 0) {
        System.out.println(1.0 / i);
    }
}
```